

# 4-Bit Custom Retro CPU Architecture Datasheet

## 1. System Overview

This is a custom software-simulated 4-Bit Microprocessor Architecture built to emulate early-generation computer hardware behavior. The design utilizes a stack-based execution cycle coupled with a dedicated instruction-decoding interface, bridging software controls with strict hardware logic constraints. [cite: 3, 4] This microprocessor utilizes a 4-bit data bus and a Stack-Based ALU architecture. [cite: 88]

## 2. Hardware Component Map

### 2.1 Storage Registers

The CPU utilizes discrete 4-bit storage arrays acting as high-speed static internal memory. [cite: 7]

- **Register AX (Program.Ax):** 4-bit general-purpose data register. [cite: 8]
- **Register BX (Program.Bx):** 4-bit general-purpose data register. [cite: 9]
- **SelectedRegister:** A hardware pointer tracking the active input state from the control unit. [cite: 10]

### 2.2 Memory Stack (Program.Stack)

A First-In, Last-Out (FILO) sequential memory array used for staging data values before execution. [cite: 12] It acts as the primary data highway between storage registers and the processing unit. [cite: 13]

- **PushRegister():** Pushes a 4-bit register state onto the stack. [cite: 14]
- **PopRegister():** Pops the top 4-bit data chunk from the stack. [cite: 15] Triggers a hardware exception (Stack Underflow) if executed on an empty stack. [cite: 16]

### 2.3 Instruction Decoder Buffer (AluInputBuffer)

An isolation latch circuit serving as a gateway to the ALU. [cite: 18] It captures incoming runtime operations and validates hardware signals before execution. [cite: 19]

- **TempA / TempB:** Internal hardware latches that isolate raw data operands. [cite: 20]
- **OpCode:** A dedicated 4-bit latch holding the operational binary command. [cite: 21]
- **Execute():** The core decoding engine that evaluates the 4-bit instruction and routes signals to the arithmetic component. [cite: 22]

## 2.4 Arithmetic Logic Unit (ALU)

The functional core of the CPU. [cite: 24] It contains pure combinational logic circuits that perform bit-level logical and arithmetic processes. [cite: 24] All arithmetic, logical, and shift operations are executed using data from the Stack. [cite: 89]

## 3. Hardware Execution Cycle (Machine Cycle)

When the EXECUTE trigger is pulsed, the CPU forces a rigid hardware cycle containing three distinct steps: [cite: 28]

1. **FETCH:** Pops Operand B, then Operand A from the Stack List. [cite: 29] Loads them into AluInputBuffer.TempB and TempA. [cite: 30]
2. **DECODE:** Captures the 4-bit Opcode from the toggle switches. [cite: 33] Evaluates the bit sequence through the buffer decoder. [cite: 34]
3. **EXECUTE:** Routes isolation latches directly into the static Alu class. [cite: 37] Calculates output, prints to OutputRegister, clears latches. [cite: 38]

*Developer Note: All internal buffers are automatically flushed via ClearBuffers() at the end of every successful execution cycle to prevent residual current/data leakage into the next clock pulse. [cite: 39]*

## 4. Schematic Component Definitions

- **4-BIT DATA BUS:** The primary highway where raw data values travel from the Input Registers (AX / BX) straight into the Stack memory. [cite: 79]
- **STACK LIST (LIFO Memory):** The data tower. [cite: 80] When executed, it automatically pops the top two data frames and latches them into TempA and TempB buffers. [cite: 80]
- **4-BIT CONTROL BUS:** This line carries the operation code signals. [cite: 81] It only carries instructions, not calculation data. [cite: 82]
- **DECODER:** Represents the AluInputBuffer.Execute() structure. [cite: 83] It reads the Control Bus, decodes the bit pattern, and activates the specific arithmetic gate inside the ALU. [cite: 83]
- **ALU:** The core mathematical engine containing all logic circuits. [cite: 85]

## 5. Crucial Architectural Rules

- **Execution Rule:** Immediately before an instruction is executed, the ALU strictly pops 2 pieces of data from the Stack (LIFO principle). [cite: 92] The first popped value is loaded into TempB, and the second popped value is loaded into TempA. [cite: 93] If the stack lacks sufficient data, the hardware automatically defaults the missing buffer(s) to 0000. [cite: 94]
- **Hardware Buffer Mapping:** 1st Popped Value (Stack Top) maps to TempB. [cite: 110]

2nd Popped Value (Stack Bottom) maps to TempA. [cite: 111]

- **Unary Instruction Execution:** Instructions requiring a single operand (e.g., SHL, SHR, INC) still force the ALU to pop 2 values from the stack. [cite: 113] To ensure the target data lands in TempA, push the target data first, followed by a "dummy" data entry. [cite: 114]
- **Binary Instruction Execution:** For operations requiring two operands, the first value pushed to the stack becomes the left side of the equation (TempA), and the second value pushed becomes the right side (TempB). [cite: 122]